1.

```
import java.util.Scanner;

public class IncomeTaxCalculator {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        double income, tax = 0, cess, totalTax;

        System.out.print("Enter Annual Income: ");
        income = sc.nextDouble();

        // Tax Calculation based on slabs
        if (income <= 400000) {
            tax = 0;
        }
        else if (income <= 800000) {
            tax = (income - 400000) * 0.05;
        }
        else if (income <= 1200000) {
            tax = (400000 * 0.05) +
                  (income - 800000) * 0.10;
        }
        else if (income <= 1600000) {
            tax = (400000 * 0.05) +
                  (400000 * 0.10) +
                  (income - 1200000) * 0.15;
        }
        else if (income <= 2000000) {
            tax = (400000 * 0.05) +
                  (400000 * 0.10) +
                  (400000 * 0.15) +
                  (income - 1600000) * 0.20;
        }
        else if (income <= 2400000) {
            tax = (400000 * 0.05) +
                  (400000 * 0.10) +
                  (400000 * 0.15) +
                  (400000 * 0.20) +
                  (income - 2000000) * 0.25;
        }
        else {
```

```
            tax = (400000 * 0.05) +
                  (400000 * 0.10) +
                  (400000 * 0.15) +
                  (400000 * 0.20) +
                  (400000 * 0.25) +
                  (income - 2400000) * 0.30;
        }

        // 4% Health and Education Cess
        cess = tax * 0.04;

        // Total Tax
        totalTax = tax + cess;

        // Display Output
        System.out.println("\n----- TAX DETAILS -----");
        System.out.println("Income Tax       : ₹" + tax);
        System.out.println("Health & Edu Cess: ₹" + cess);
        System.out.println("Total Tax Payable: ₹" + totalTax);

        sc.close();
    }
}
```

_____&&&&&&_____

2.

```
import java.util.Scanner;

class Fraction {
    int numerator;
    int denominator;

    // Constructor
    Fraction(int n, int d) {
        numerator = n;
        denominator = d;
    }

    // Method to add two fractions
    Fraction add(Fraction f2) {
        int num = (numerator * f2.denominator) +
                  (f2.numerator * denominator);
```

```
        int den = denominator * f2.denominator;

        int gcdValue = gcd(num, den);

        num = num / gcdValue;
        den = den / gcdValue;

        return new Fraction(num, den);
    }

    // Method to find GCD
    int gcd(int a, int b) {
        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }
        return a;
    }

    // Display fraction
    void display() {
        System.out.println("Total Sugar Required = "
                           + numerator + "/" + denominator);
    }
}

public class FractionAddition {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // First Fraction
        System.out.print("Enter numerator of first fraction: ");
        int n1 = sc.nextInt();

        System.out.print("Enter denominator of first fraction: ");
        int d1 = sc.nextInt();

        // Second Fraction
        System.out.print("Enter numerator of second fraction: ");
        int n2 = sc.nextInt();
```

```
        System.out.print("Enter denominator of second fraction: ");
        int d2 = sc.nextInt();

        Fraction f1 = new Fraction(n1, d1);
        Fraction f2 = new Fraction(n2, d2);

        Fraction result = f1.add(f2);

        result.display();

        sc.close();
    }
}
```

_____&&&&&&_____

3.

```
import java.util.Scanner;
import java.util.Arrays;

public class ArrayEquality {

    // Method without predefined function
    static boolean manualCheck(int arr1[], int arr2[]) {

        if (arr1.length != arr2.length) {
            return false;
        }

        for (int i = 0; i < arr1.length; i++) {
            if (arr1[i] != arr2[i]) {
                return false;
            }
        }

        return true;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter size of arrays: ");
```

```
        int n = sc.nextInt();

        int arr1[] = new int[n];
        int arr2[] = new int[n];

        // Input first array
        System.out.println("Enter elements of first array:");
        for (int i = 0; i < n; i++) {
            arr1[i] = sc.nextInt();
        }

        // Input second array
        System.out.println("Enter elements of second array:");
        for (int i = 0; i < n; i++) {
            arr2[i] = sc.nextInt();
        }

        // Manual method
        if (manualCheck(arr1, arr2)) {
            System.out.println("\nManual Method: Arrays are Equal");
        } else {
            System.out.println("\nManual Method: Arrays are Not Equal");
        }

        // Predefined method
        if (Arrays.equals(arr1, arr2)) {
            System.out.println("Using Arrays.equals(): Arrays are Equal");
        } else {
            System.out.println("Using Arrays.equals(): Arrays are Not Equal");
        }

        sc.close();
    }
}
```

_____&&&&&&_____

4.

```
import java.util.Scanner;

class Overload {
```

```
    // Sum of 2 integers
    int sum(int a, int b) {
        return a + b;
    }

    // Sum of 3 integers
    int sum(int a, int b, int c) {
        return a + b + c;
    }

    // Sum of array elements
    int sum(int arr[]) {
        int total = 0;

        for (int i = 0; i < arr.length; i++) {
            total = total + arr[i];
        }

        return total;
    }

    // Sum of 2 double values
    double sum(double a, double b) {
        return a + b;
    }
}

public class MethodOverloading {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Overload obj = new Overload();

        // Sum of 2 integers
        System.out.print("Enter two integers: ");
        int a = sc.nextInt();
        int b = sc.nextInt();

        System.out.println("Sum of 2 integers = "
                           + obj.sum(a, b));

        // Sum of 3 integers
```

```
        System.out.print("\nEnter three integers: ");
        int x = sc.nextInt();
        int y = sc.nextInt();
        int z = sc.nextInt();

        System.out.println("Sum of 3 integers = "
                        + obj.sum(x, y, z));

        // Sum of array elements
        System.out.print("\nEnter size of array: ");
        int n = sc.nextInt();

        int arr[] = new int[n];

        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        System.out.println("Sum of array elements = "
                        + obj.sum(arr));

        // Sum of 2 double values
        System.out.print("\nEnter two double values: ");
        double d1 = sc.nextDouble();
        double d2 = sc.nextDouble();

        System.out.println("Sum of double values = "
                        + obj.sum(d1, d2));

        sc.close();
    }
}
```

_____&&&&&&_____

5.

```
package calculator;

public class Operations {

    public int add(int a, int b) {
        return a + b;
```

```
    }

    public int sub(int a, int b) {
        return a - b;
    }

    public int mul(int a, int b) {
        return a * b;
    }

    public int div(int a, int b) {
        return a / b;
    }

    public int mod(int a, int b) {
        return a % b;
    }
}



import java.util.Scanner;
import calculator.Operations;

public class CalculatorApp {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Operations obj = new Operations();

        System.out.print("Enter first number: ");
        int a = sc.nextInt();

        System.out.print("Enter second number: ");
        int b = sc.nextInt();

        System.out.println("\n--- RESULTS ---");

        System.out.println("Addition = " + obj.add(a, b));

        System.out.println("Subtraction = " + obj.sub(a, b));
```

```
        System.out.println("Multiplication = " + obj.mul(a, b));

        System.out.println("Division = " + obj.div(a, b));

        System.out.println("Modulus = " + obj.mod(a, b));

        sc.close();
    }
}
```

_____&&&&&&_____

6.

```
package MCA;

public class Student {

    String name;
    int rollNo;
    int mark1, mark2, mark3;

    // Parameterized Constructor
    public Student(String name, int rollNo,
                   int mark1, int mark2, int mark3) {

        this.name = name;
        this.rollNo = rollNo;
        this.mark1 = mark1;
        this.mark2 = mark2;
        this.mark3 = mark3;
    }

    // Display Method
    public void display() {

        System.out.println("\n--- STUDENT DETAILS ---");

        System.out.println("Name       : " + name);
        System.out.println("Roll No    : " + rollNo);
        System.out.println("Mark 1     : " + mark1);
        System.out.println("Mark 2     : " + mark2);
```

```
        System.out.println("Mark 3     : " + mark3);
    }
}
```

```
import java.util.Scanner;
import MCA.Student;

public class StudentMain {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Input Student Details
        System.out.print("Enter Student Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Roll Number: ");
        int rollNo = sc.nextInt();

        System.out.print("Enter Mark 1: ");
        int m1 = sc.nextInt();

        System.out.print("Enter Mark 2: ");
        int m2 = sc.nextInt();

        System.out.print("Enter Mark 3: ");
        int m3 = sc.nextInt();

        // Object Creation
        Student obj = new Student(name, rollNo, m1, m2, m3);

        // Display Details
        obj.display();

        // Calculate Total and Percentage
        int total = m1 + m2 + m3;

        double percentage = total / 3.0;

        System.out.println("Total Marks : " + total);
```

```
        System.out.println("Percentage  : "
                        + percentage + "%");

        sc.close();
    }
}
```

_____&&&&&&_____

7.

```
import java.util.Scanner;

// Base Class
class Employee {

    String empName;
    int empId;
    String address;
    String mailId;
    String mobileNo;

    // Method to get common details
    void getEmployeeDetails() {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Employee Name: ");
        empName = sc.nextLine();

        System.out.print("Enter Employee ID: ");
        empId = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Address: ");
        address = sc.nextLine();

        System.out.print("Enter Mail ID: ");
        mailId = sc.nextLine();

        System.out.print("Enter Mobile Number: ");
        mobileNo = sc.nextLine();
    }
```

```
    // Display Common Details
    void displayEmployeeDetails() {

        System.out.println("Employee Name : " + empName);
        System.out.println("Employee ID   : " + empId);
        System.out.println("Address       : " + address);
        System.out.println("Mail ID       : " + mailId);
        System.out.println("Mobile No     : " + mobileNo);
    }
}

// Derived Class
class Programmer extends Employee {

    double BP, DA, HRA, PF, staffClubFund;
    double grossSalary, netSalary;

    void getBP() {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Basic Pay: ");
        BP = sc.nextDouble();
    }

    void calculateSalary() {

        DA = 0.97 * BP;

        HRA = 0.10 * BP;

        PF = 0.12 * BP;

        staffClubFund = 0.001 * BP;

        grossSalary = BP + DA + HRA;

        netSalary = grossSalary - PF - staffClubFund;
    }

    void displayPaySlip() {

        System.out.println("\n========== PAY SLIP ==========");
```

```
        System.out.println("Designation   : Programmer");

        displayEmployeeDetails();

        System.out.println("Basic Pay     : " + BP);

        System.out.println("DA            : " + DA);

        System.out.println("HRA           : " + HRA);

        System.out.println("PF            : " + PF);

        System.out.println("Staff Club Fund : "
                + staffClubFund);

        System.out.println("Gross Salary  : "
                + grossSalary);

        System.out.println("Net Salary    : "
                + netSalary);

        System.out.println("==============================");
    }
}

// Other Derived Classes
class AssistantProfessor extends Employee {

    double BP;
}

class AssociateProfessor extends Employee {

    double BP;
}

class Professor extends Employee {

    double BP;
}

// Main Class
public class PayrollManagement {
```

```
    public static void main(String[] args) {

        Programmer obj = new Programmer();

        obj.getEmployeeDetails();

        obj.getBP();

        obj.calculateSalary();

        obj.displayPaySlip();
    }
}
```

_____&&&&&&_____

8.

```
import java.util.Scanner;

// Interface
interface AdvancedArithmetic {

    int sum(int a, int b);

    int sub(int a, int b);

    int mul(int a, int b);

    int div(int a, int b);

    int divisorSum(int n);
}

// Implementing Class
class MyCalculator implements AdvancedArithmetic {

    // Addition
    public int sum(int a, int b) {
        return a + b;
    }
```

```
    // Subtraction
    public int sub(int a, int b) {
        return a - b;
    }

    // Multiplication
    public int mul(int a, int b) {
        return a * b;
    }

    // Division
    public int div(int a, int b) {
        return a / b;
    }

    // Sum of divisors
    public int divisorSum(int n) {

        int sum = 0;

        for (int i = 1; i <= n; i++) {

            if (n % i == 0) {
                sum = sum + i;
            }
        }

        return sum;
    }
}

// Main Class
public class InterfaceProgram {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        MyCalculator obj = new MyCalculator();

        System.out.print("Enter first number: ");
        int a = sc.nextInt();

        System.out.print("Enter second number: ");
```

```
        int b = sc.nextInt();

        System.out.println("\n--- Arithmetic Operations ---");

        System.out.println("Addition = "
                        + obj.sum(a, b));

        System.out.println("Subtraction = "
                        + obj.sub(a, b));

        System.out.println("Multiplication = "
                        + obj.mul(a, b));

        System.out.println("Division = "
                        + obj.div(a, b));

        System.out.print("\nEnter number to find sum of divisors: ");

        int n = sc.nextInt();

        System.out.println("Sum of Divisors = "
                        + obj.divisorSum(n));

        sc.close();
    }
}
```

_____&&&&&&_____

9.

```
import java.util.Arrays;

class AnagramChecker {

    boolean areAnagrams(String str1, String str2) {

        // Remove spaces and special characters
        str1 = str1.replaceAll("[^a-zA-Z]", "").toLowerCase();

        str2 = str2.replaceAll("[^a-zA-Z]", "").toLowerCase();

        // Convert strings to character arrays
```

```
        char arr1[] = str1.toCharArray();

        char arr2[] = str2.toCharArray();

        // Sort arrays
        Arrays.sort(arr1);

        Arrays.sort(arr2);

        // Compare arrays
        return Arrays.equals(arr1, arr2);
    }
}

public class Main {

    public static void main(String[] args) {

        AnagramChecker obj = new AnagramChecker();

        String s1 = "rail safety";

        String s2 = "fairy tales";

        if (obj.areAnagrams(s1, s2)) {

            System.out.println("The given strings are Anagrams");
        }
        else {

            System.out.println("The given strings are Not Anagrams");
        }
    }
}
```

_____&&&&&&_____

10.

```
import java.util.Scanner;

// User Defined Exception
class NegativeValueException extends Exception {
```

```
    NegativeValueException(String message) {

        super(message);
    }
}

public class NegativeArray {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter size of array: ");

        int n = sc.nextInt();

        int arr[] = new int[n];

        System.out.println("Enter array elements:");

        for (int i = 0; i < n; i++) {

            try {

                int value = sc.nextInt();

                // Check negative value
                if (value < 0) {

                    throw new NegativeValueException(
                        "Negative values are not allowed"
                    );
                }

                arr[i] = value;
            }

            catch (NegativeValueException e) {

                System.out.println("Exception: "
                                   + e.getMessage());

                i--; // Re-enter valid input
```

```
            }
        }

        // Display array
        System.out.println("\nValid Array Elements:");

        for (int i = 0; i < n; i++) {

            System.out.print(arr[i] + " ");
        }

        sc.close();
    }
}
```

_____&&&&&&_____

11.

```
import java.io.*;

public class FileProgram {

    public static void main(String[] args) {

        try {

            FileReader fr = new FileReader("source.txt");

            BufferedReader br = new BufferedReader(fr);

            FileWriter fw = new FileWriter("destination.txt");

            BufferedWriter bw = new BufferedWriter(fw);

            String line;

            while ((line = br.readLine()) != null) {

                String words[] = line.split(" ");

                for (int i = 0; i < words.length; i++) {
```

```
                String word = words[i];

                // Convert first letter to uppercase
                String newWord =
                   Character.toUpperCase(word.charAt(0))
                   + word.substring(1);

                bw.write(newWord + " ");
             }

             bw.newLine();
          }

          br.close();

          bw.close();

          System.out.println("Data copied successfully");
       }

       catch (Exception e) {

          System.out.println(e);
       }
    }
}
```

_____&&&&&&_____

12.

```
import java.util.Random;

// Thread for generating numbers
class NumberGenerator extends Thread {

    public void run() {

        Random r = new Random();

        int evenCount = 0;
        int oddCount = 0;
```

```
        for (int i = 1; i <= 10; i++) {

            int num = r.nextInt(100);

            System.out.println("\nGenerated Number: " + num);

            // Even number
            if (num % 2 == 0) {

                EvenThread e = new EvenThread(num);

                e.start();

                evenCount++;
            }

            // Odd number
            else {

                OddThread o = new OddThread(num);

                o.start();

                oddCount++;
            }

            System.out.println("Even Count = " + evenCount);

            System.out.println("Odd Count = " + oddCount);

            try {

                Thread.sleep(1000);
            }

            catch (Exception e) {

                System.out.println(e);
            }
        }
    }
}

// Thread for even numbers
```

```
class EvenThread extends Thread {

    int num;

    EvenThread(int num) {

        this.num = num;
    }

    public void run() {

        System.out.println(
            "Square of " + num + " = "
            + (num * num));
    }
}

// Thread for odd numbers
class OddThread extends Thread {

    int num;

    OddThread(int num) {

        this.num = num;
    }

    public void run() {

        System.out.println(
            "Cube of " + num + " = "
            + (num * num * num));
    }
}

// Main Class
public class MultiThreadProgram {

    public static void main(String[] args) {

        NumberGenerator obj = new NumberGenerator();

        obj.start();
    }
```

```
}
```

_____&&&&&&_____

13.

```
import java.util.Random;

// Thread to generate numbers
class NumberGenerator extends Thread {

    public void run() {

        Random r = new Random();

        int factorialCount = 0;

        int digitSumCount = 0;

        for (int i = 1; i <= 10; i++) {

            int num = r.nextInt(20) + 1;

            System.out.println("\nGenerated Number: " + num);

            // Divisible by 3
            if (num % 3 == 0) {

                FactorialThread f =
                    new FactorialThread(num);

                f.start();

                factorialCount++;
            }

            // Not divisible by 3
            else {

                DigitSumThread d =
                    new DigitSumThread(num);

                d.start();
```

```
            digitSumCount++;
        }

        System.out.println(
            "Factorial Count = "
            + factorialCount);

        System.out.println(
            "Digit Sum Count = "
            + digitSumCount);

        try {

            Thread.sleep(2000);
        }

        catch (Exception e) {

            System.out.println(e);
        }
    }
  }
}

// Thread for factorial
class FactorialThread extends Thread {

    int num;

    FactorialThread(int num) {

        this.num = num;
    }

    public void run() {

        int fact = 1;

        for (int i = 1; i <= num; i++) {

            fact = fact * i;
        }
```

```
        System.out.println(
            "Factorial of " + num
            + " = " + fact);
    }
}

// Thread for sum of digits
class DigitSumThread extends Thread {

    int num;

    DigitSumThread(int num) {

        this.num = num;
    }

    public void run() {

        int temp = num;

        int sum = 0;

        while (temp > 0) {

            sum = sum + temp % 10;

            temp = temp / 10;
        }

        System.out.println(
            "Sum of digits of " + num
            + " = " + sum);
    }
}

// Main Class
public class ThreadProgram {

    public static void main(String[] args) {

        NumberGenerator obj =
            new NumberGenerator();

        obj.start();
```

```
    }
}
```

_____&&&&&&_____

14.

```
class Thread1 extends Thread {
    public void run() {
        try {
            for (char ch = 'A'; ch <= 'Z'; ch++) {
                System.out.print(ch + " ");
                Thread.sleep(1000);
            }
        }
        catch (Exception e) {
            System.out.println(e);
        }
    }
}
class Thread2 extends Thread {
    public void run() {
        try {
            for (char ch = 'Z'; ch >= 'A'; ch--) {
                System.out.print(ch + " ");
                Thread.sleep(2000);
            }
        }
```

```
        catch (Exception e) {

            System.out.println(e);
        }
    }
}

public class AlphabetThreads {

    public static void main(String[] args) {

        Thread1 t1 = new Thread1();

        Thread2 t2 = new Thread2();

        // Start first thread
        t1.start();

        try {

            // Wait until Thread1 finishes
            t1.join();
        }

        catch (Exception e) {

            System.out.println(e);
        }

        // Start second thread
        t2.start();

        try {

            // Wait until Thread2 finishes
            t2.join();
        }

        catch (Exception e) {

            System.out.println(e);
        }

        System.out.println("\nBoth Threads Finished");
```

```
    }
}
```

_____&&&&&&_____

18.

```
import java.util.Scanner;
import java.util.Arrays;

public class ArrayOperations {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // First Array
        System.out.print("Enter size of first array: ");
        int n1 = sc.nextInt();

        int arr1[] = new int[n1];

        System.out.println("Enter first array elements:");

        for (int i = 0; i < n1; i++) {

            arr1[i] = sc.nextInt();
        }

        // Second Array
        System.out.print("\nEnter size of second array: ");
        int n2 = sc.nextInt();

        int arr2[] = new int[n2];

        System.out.println("Enter second array elements:");

        for (int i = 0; i < n2; i++) {

            arr2[i] = sc.nextInt();
        }

        // Merge arrays manually
```

```
int merged[] = new int[n1 + n2];

int k = 0;

for (int i = 0; i < n1; i++) {

    merged[k++] = arr1[i];
}

for (int i = 0; i < n2; i++) {

    merged[k++] = arr2[i];
}

// Remove duplicates manually
int unique[] = new int[merged.length];

int size = 0;

for (int i = 0; i < merged.length; i++) {

    boolean isDuplicate = false;

    for (int j = 0; j < size; j++) {

        if (merged[i] == unique[j]) {

            isDuplicate = true;

            break;
        }
    }

    if (!isDuplicate) {

        unique[size++] = merged[i];
    }
}

// Copy unique elements
int result[] = new int[size];

for (int i = 0; i < size; i++) {
```

```
            result[i] = unique[i];
        }

        // Sort array
        Arrays.sort(result);

        // Display sorted unique array
        System.out.println("\nSorted Unique Array:");

        for (int i = 0; i < result.length; i++) {

            System.out.print(result[i] + " ");
        }

        // Find sum, max, min
        int sum = 0;

        int max = result[0];

        int min = result[0];

        for (int i = 0; i < result.length; i++) {

            sum = sum + result[i];

            if (result[i] > max) {

                max = result[i];
            }

            if (result[i] < min) {

                min = result[i];
            }
        }

        System.out.println("\n\nSum = " + sum);

        System.out.println("Maximum = " + max);

        System.out.println("Minimum = " + min);

        sc.close();
    }
```

```
}
```

_____&&&&&&_____

20. SAME

```
package scientific;

public class Operations {

    // Power
    public double power(int x, int y) {

        return Math.pow(x, y);
    }

    // Square Root
    public double squareRoot(int n) {

        return Math.sqrt(n);
    }

    // Factorial
    public int factorial(int n) {

        int fact = 1;

        for (int i = 1; i <= n; i++) {

            fact = fact * i;
        }

        return fact;
    }

    // GCD
    public int gcd(int a, int b) {

        while (b != 0) {

            int temp = b;

            b = a % b;
```

```
            a = temp;
        }

        return a;
    }
}
```

_____&&&&&&_____

21.SAME 6

```
package StudentMgmt;

public class Student {

    String name;

    int rollNo;

    int marks[] = new int[5];

    int total;

    double average;

    String grade;

    // Constructor
    public Student(String name,
                   int rollNo,
                   int marks[]) {

        this.name = name;

        this.rollNo = rollNo;

        this.marks = marks;
    }

    // Calculate Total
    public void calculateTotal() {
```

```
    total = 0;

    for (int i = 0; i < marks.length; i++) {

        total = total + marks[i];
    }
}

// Calculate Average
public void calculateAverage() {

    average = total / 5.0;
}

// Assign Grade
public void calculateGrade() {

    if (average >= 90) {

        grade = "A";
    }

    else if (average >= 75) {

        grade = "B";
    }

    else if (average >= 50) {

        grade = "C";
    }

    else {

        grade = "Fail";
    }
}

// Display Result
public void display() {

    System.out.println("\n--- STUDENT RESULT ---");
```

```
        System.out.println("Name       : " + name);
        System.out.println("Roll No    : " + rollNo);
        System.out.println("Total      : " + total);
        System.out.println("Average    : " + average);
        System.out.println("Grade      : " + grade);
    }
}
```

```
import java.util.Scanner;
import StudentMgmt.Student;
public class StudentMain {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Student Name: ");
        String name = sc.nextLine();
        System.out.print("Enter Roll Number: ");
        int rollNo = sc.nextInt();
        int marks[] = new int[5];
        System.out.println("Enter marks of 5 subjects:");
        for (int i = 0; i < 5; i++) {
            marks[i] = sc.nextInt();
        }
        // Object Creation
```

```
        Student obj =
            new Student(name, rollNo, marks);

        // Method Calls
        obj.calculateTotal();

        obj.calculateAverage();

        obj.calculateGrade();

        obj.display();

        sc.close();
    }
}
```

_____&&&&&&_____

22.SAME 7

```
import java.util.Scanner;

// Base Class
class Employee {

    String name;

    int empId;

    String designation;

    void getEmployeeDetails() {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Employee Name: ");

        name = sc.nextLine();

        System.out.print("Enter Employee ID: ");

        empId = sc.nextInt();
```

```
        sc.nextLine();

        System.out.print("Enter Designation: ");

        designation = sc.nextLine();
    }

    void displayEmployeeDetails() {

        System.out.println("Employee Name : " + name);

        System.out.println("Employee ID   : " + empId);

        System.out.println("Designation   : "
                           + designation);
    }
}

// Derived Class
class Manager extends Employee {

    double BP, DA, HRA, PF;
    double grossSalary, netSalary, tax;

    void getBP() {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Basic Pay: ");

        BP = sc.nextDouble();
    }

    void calculateSalary() {

        DA = 0.97 * BP;

        HRA = 0.10 * BP;

        PF = 0.12 * BP;

        grossSalary = BP + DA + HRA;
```

```
        tax = 0.10 * grossSalary;

        netSalary = grossSalary - PF - tax;
    }

    void displayPaySlip() {

        System.out.println("\n===== PAY SLIP =====");

        displayEmployeeDetails();

        System.out.println("Basic Pay    : " + BP);

        System.out.println("DA           : " + DA);

        System.out.println("HRA          : " + HRA);

        System.out.println("PF           : " + PF);

        System.out.println("Tax          : " + tax);

        System.out.println("Gross Salary : "
                + grossSalary);

        System.out.println("Net Salary   : "
                + netSalary);
    }
}

// Other Derived Classes
class Engineer extends Employee {

    double BP;
}

class Clerk extends Employee {

    double BP;
}

// Main Class
public class PayrollSystem {

    public static void main(String[] args) {
```

```
        Manager obj = new Manager();

        obj.getEmployeeDetails();

        obj.getBP();

        obj.calculateSalary();

        obj.displayPaySlip();
    }
}
```

_____&&&&&&_____

23.

```
import java.util.Scanner;

// Interface
interface Payment {

    void pay(double amount);
}

// Credit Card Class
class CreditCard implements Payment {

    double balance = 5000;

    public void pay(double amount) {

        if (amount <= balance) {

            balance = balance - amount;

            int transactionId =
                (int)(Math.random() * 10000);

            System.out.println("\nPayment Method : Credit Card");

            System.out.println("Amount Paid    : "
```

```
                    + amount);

            System.out.println("Remaining Balance : "
                    + balance);

            System.out.println("Transaction ID : "
                    + transactionId);
        }

        else {

            System.out.println(
                "Insufficient Balance");
        }
    }
}

// Debit Card Class
class DebitCard implements Payment {

    double balance = 4000;

    public void pay(double amount) {

        if (amount <= balance) {

            balance = balance - amount;

            int transactionId =
                (int)(Math.random() * 10000);

            System.out.println("\nPayment Method : Debit Card");

            System.out.println("Amount Paid    : "
                    + amount);

            System.out.println("Remaining Balance : "
                    + balance);

            System.out.println("Transaction ID : "
                    + transactionId);
        }

        else {
```

```
            System.out.println(
                "Insufficient Balance");
        }
    }
}

// UPI Class
class UPI implements Payment {

    double balance = 3000;

    public void pay(double amount) {

        if (amount <= balance) {

            balance = balance - amount;

            int transactionId =
                (int)(Math.random() * 10000);

            System.out.println("\nPayment Method : UPI");

            System.out.println("Amount Paid    : "
                               + amount);

            System.out.println("Remaining Balance : "
                               + balance);

            System.out.println("Transaction ID : "
                               + transactionId);
        }

        else {

            System.out.println(
                "Insufficient Balance");
        }
    }
}

// Net Banking Class
class NetBanking implements Payment {
```

```
    double balance = 6000;

    public void pay(double amount) {

        if (amount <= balance) {

            balance = balance - amount;

            int transactionId =
                (int)(Math.random() * 10000);

            System.out.println("\nPayment Method : Net Banking");

            System.out.println("Amount Paid    : "
                               + amount);

            System.out.println("Remaining Balance : "
                               + balance);

            System.out.println("Transaction ID : "
                               + transactionId);
        }

        else {

            System.out.println(
                "Insufficient Balance");
        }
    }
}

// Wallet Class
class Wallet implements Payment {

    double balance = 2000;

    public void pay(double amount) {

        if (amount <= balance) {

            balance = balance - amount;

            int transactionId =
                (int)(Math.random() * 10000);
```

```
            System.out.println("\nPayment Method : Wallet");

            System.out.println("Amount Paid    : "
                               + amount);

            System.out.println("Remaining Balance : "
                               + balance);

            System.out.println("Transaction ID : "
                               + transactionId);
        }

        else {

            System.out.println(
                "Insufficient Balance");
        }
    }
}

// Main Class
public class PaymentSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Payment p;

        System.out.println("1. Credit Card");

        System.out.println("2. Debit Card");

        System.out.println("3. UPI");

        System.out.println("4. Net Banking");

        System.out.println("5. Wallet");

        System.out.print("Choose Payment Method: ");

        int choice = sc.nextInt();
```

```
        System.out.print("Enter Amount: ");

        double amount = sc.nextDouble();

        switch (choice) {

            case 1:
                p = new CreditCard();
                break;

            case 2:
                p = new DebitCard();
                break;

            case 3:
                p = new UPI();
                break;

            case 4:
                p = new NetBanking();
                break;

            case 5:
                p = new Wallet();
                break;

            default:
                System.out.println("Invalid Choice");
                return;
        }

        // Runtime Polymorphism
        p.pay(amount);

        sc.close();
    }
}
```

Since polymorphism I have created object for interface else I would have created object for class as did in question 7

_____&&&&&&_____

24.SAME 9

Here anagrams and palindrame

```
import java.util.Arrays;
import java.util.Scanner;

public class StringCheck {

    // Method to process string
    static String process(String str) {

        str = str.replaceAll("[^a-zA-Z]", "")
                .toLowerCase();

        return str;
    }

    // Method to check palindrome
    static boolean isPalindrome(String str) {

        String rev = "";

        for (int i = str.length() - 1; i >= 0; i--) {

            rev = rev + str.charAt(i);
        }

        return str.equals(rev);
    }

    // Method to check anagram
    static boolean isAnagram(String s1, String s2) {

        char arr1[] = s1.toCharArray();

        char arr2[] = s2.toCharArray();

        Arrays.sort(arr1);

        Arrays.sort(arr2);

        return Arrays.equals(arr1, arr2);
```

```
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter first string: ");

    String str1 = sc.nextLine();

    System.out.print("Enter second string: ");

    String str2 = sc.nextLine();

    // Process strings
    str1 = process(str1);

    str2 = process(str2);

    // Anagram check
    if (isAnagram(str1, str2)) {

        System.out.println(
            "\nStrings are Anagrams");
    }

    else {

        System.out.println(
            "\nStrings are Not Anagrams");
    }

    // Palindrome check for first string
    if (isPalindrome(str1)) {

        System.out.println(
            "First string is Palindrome");
    }

    else {

        System.out.println(
            "First string is Not Palindrome");
    }
```

```
        // Palindrome check for second string
        if (isPalindrome(str2)) {

            System.out.println(
                "Second string is Palindrome");
        }

        else {

            System.out.println(
                "Second string is Not Palindrome");
        }

        sc.close();
    }
}
```

_____&&&&&&_____

25.

```
import java.util.Scanner;

class StringOperations {

    // Count vowels
    int countVowels(String str) {

        int count = 0;

        str = str.toLowerCase();

        for (int i = 0; i < str.length(); i++) {

            char ch = str.charAt(i);

            if (ch == 'a' || ch == 'e' ||
                ch == 'i' || ch == 'o' ||
                ch == 'u') {

                count++;
```

```
        }
    }

    return count;
}

// Check palindrome
boolean isPalindrome(String str) {

    String rev = "";

    for (int i = str.length() - 1; i >= 0; i--) {

        rev = rev + str.charAt(i);
    }

    return str.equalsIgnoreCase(rev);
}

// Reverse string
String reverse(String str) {

    String rev = "";

    for (int i = str.length() - 1; i >= 0; i--) {

        rev = rev + str.charAt(i);
    }

    return rev;
}

// Count words
int countWords(String str) {

    String words[] = str.split(" ");

    return words.length;
}

// Remove duplicate characters
String removeDuplicates(String str) {

    String result = "";
```

```
        for (int i = 0; i < str.length(); i++) {

            char ch = str.charAt(i);

            if (result.indexOf(ch) == -1) {

                result = result + ch;
            }
        }

        return result;
    }
}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");

        String input = sc.nextLine();

        StringOperations obj =
            new StringOperations();

        System.out.println("\nVowel Count = "
                           + obj.countVowels(input));

        if (obj.isPalindrome(input)) {

            System.out.println(
                "String is Palindrome");
        }

        else {

            System.out.println(
                "String is Not Palindrome");
        }

        System.out.println("Reversed String = "
```

```
                + obj.reverse(input));

        System.out.println("Word Count = "
                + obj.countWords(input));

        System.out.println("After Removing Duplicates = "
                + obj.removeDuplicates(input));

        sc.close();
    }
}
```

_____&&&&&&_____

26.

```
import java.util.Scanner;

class SimpleStringOperations {

    // Length of string
    int lengthOfString(String str) {

        return str.length();
    }

    // Convert to uppercase
    String toUpperCase(String str) {

        StringBuilder a = new StringBuilder(str);

        for (int i = 0; i < a.length(); i++) {

            if (a.charAt(i) >= 97 &&
                a.charAt(i) <= 122) {

                int c = (int)a.charAt(i) - 32;

                a.setCharAt(i, (char)c);
            }
        }
```

```
    return a.toString();
}

// Convert to lowercase
String toLowerCase(String str) {

    StringBuilder a = new StringBuilder(str);

    for (int i = 0; i < a.length(); i++) {

        if (a.charAt(i) >= 65 &&
            a.charAt(i) <= 90) {

            int c = (int)a.charAt(i) + 32;

            a.setCharAt(i, (char)c);
        }
    }

    return a.toString();
}

// Reverse string
String reverseString(String str) {

    String rev = "";

    for (int i = str.length() - 1; i >= 0; i--) {

        rev = rev + str.charAt(i);
    }

    return rev;
}

// Count spaces
int countSpaces(String str) {

    int count = 0;

    for (int i = 0; i < str.length(); i++) {

        if (str.charAt(i) == ' ') {
```

```
            count++;
        }
      }

      return count;
   }
}

public class Main {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a string: ");

      String input = sc.nextLine();

      SimpleStringOperations obj =
         new SimpleStringOperations();

      System.out.println("\nLength = "
                  + obj.lengthOfString(input));

      System.out.println("Uppercase = "
                  + obj.toUpperCase(input));

      System.out.println("Lowercase = "
                  + obj.toLowerCase(input));

      System.out.println("Reversed String = "
                  + obj.reverseString(input));

      System.out.println("Space Count = "
                  + obj.countSpaces(input));

      sc.close();
   }
}
```

_____&&&&&&_____

27.

```
import java.util.Scanner;

// User Defined Exception
class InvalidAgeException extends Exception {

    InvalidAgeException(String message) {

        super(message);
    }
}

public class VotingEligibility {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Age: ");

        int age = sc.nextInt();

        try {

            // Check age
            if (age < 18) {

                throw new InvalidAgeException(
                    "Not Eligible for Voting"
                );
            }

            System.out.println(
                "Eligible for Voting");
        }

        catch (InvalidAgeException e) {

            System.out.println(
                "Exception: "
                + e.getMessage());
        }
```

```
        sc.close();
    }
}
```

_____&&&&&&_____

28.

```
import java.io.*;

public class FileAnalysis {

    public static void main(String[] args) {

        try {

            // Reading file
            FileReader fr =
                new FileReader("input.txt");

            BufferedReader br =
                new BufferedReader(fr);

            // Writing file
            FileWriter fw =
                new FileWriter("output.txt");

            BufferedWriter bw =
                new BufferedWriter(fw);

            String line;

            int lineCount = 0;

            int wordCount = 0;

            int charCount = 0;

            while ((line = br.readLine()) != null) {

                lineCount++;

                charCount =
```

```
                charCount + line.length();

            String words[] = line.split(" ");

            wordCount =
                wordCount + words.length;
        }

        // Writing results
        bw.write("Number of Lines : "
                + lineCount);

        bw.newLine();

        bw.write("Number of Words : "
                + wordCount);

        bw.newLine();

        bw.write("Number of Characters : "
                + charCount);

        br.close();

        bw.close();

        System.out.println(
            "Results written successfully");
      }

      catch (Exception e) {

        System.out.println(e);
      }
   }
}
```

_____&&&&&&_____

29.

```
import java.util.Random;
```

```
// Thread to generate numbers
class NumberGenerator extends Thread {

    public void run() {

        Random r = new Random();

        int primeCount = 0;

        int nonPrimeCount = 0;

        for (int i = 1; i <= 10; i++) {

            int num = r.nextInt(20) + 1;

            System.out.println(
                "\nGenerated Number: " + num);

            if (isPrime(num)) {

                PrimeThread p =
                    new PrimeThread(num);

                p.start();

                primeCount++;
            }

            else {

                NonPrimeThread n =
                    new NonPrimeThread(num);

                n.start();

                nonPrimeCount++;
            }

            System.out.println(
                "Prime Count = "
                + primeCount);

            System.out.println(
                "Non-Prime Count = "
```

```
            + nonPrimeCount);

            try {

                Thread.sleep(1000);
            }

            catch (Exception e) {

                System.out.println(e);
            }
        }
    }

    // Prime check method
    boolean isPrime(int num) {

        if (num <= 1) {

            return false;
        }

        for (int i = 2; i < num; i++) {

            if (num % i == 0) {

                return false;
            }
        }

        return true;
    }
}

// Thread for prime numbers
class PrimeThread extends Thread {

    int num;

    PrimeThread(int num) {

        this.num = num;
    }
```

```
    public void run() {

        System.out.println(
            "Prime Number : " + num);
    }
}

// Thread for non-prime numbers
class NonPrimeThread extends Thread {

    int num;

    NonPrimeThread(int num) {

        this.num = num;
    }

    public void run() {

        System.out.print(
            "Factors of " + num + " : ");

        for (int i = 1; i <= num; i++) {

            if (num % i == 0) {

                System.out.print(i + " ");
            }
        }

        System.out.println();
    }
}

// Main Class
public class PrimeThreadProgram {

    public static void main(String[] args) {

        NumberGenerator obj =
            new NumberGenerator();

        obj.start();
    }
```

```
}
```